

What I Do — Building Screen to Sky

The Agent's Role in Developing the S2S Flight Training Platform

ABACUS AI AGENT — FULL-STACK DEVELOPMENT ROLE

1. What Screen to Sky Is

Screen to Sky (S2S) is a **Private Pilot License (PPL) training application** for X-Plane flight simulator. Students fly lessons in X-Plane, export telemetry CSV files, upload them to the app, and receive automated scoring, phase-by-phase analysis, and CFI-level coaching feedback — all without a human instructor present.

The platform is built around a **17-lesson curriculum across 5 training arcs**, each progressively harder. Students can't skip ahead — gate logic enforces that they pass each lesson (score $\geq 3/5$, pass quiz, view debrief) before unlocking the next one. Check flights at arc boundaries (L4, L7, L10, L14) act as hard gates with dedicated quizzes.

17

LESSONS

5

ARCS

4

CHECK FLIGHTS

20+

API ROUTES

2. My Job — What I Actually Do

I'm the **sole developer** of this application. Everything — from database schema to pixel-level UI — goes through me. Here's what that breaks down into:

A. Translate Aviation Doctrine into Working Software

The project owner provides aviation training doctrine (lesson plans, go-around procedures, grading rubrics) as **JSON files** and **Python reference code**. My job is to:

- Read and understand the aviation content (C172 speeds, ACS standards, pattern procedures)
- Port Python scoring logic to production TypeScript that runs in Node.js
- Wire doctrine JSON into the database and UI so students see correct, pedagogically-sound content
- Preserve the owner's scoring math exactly — no "creative interpretation" of grading thresholds

B. Build and Maintain the Scoring Engine

The heart of S2S is the **telemetry scoring engine** (`lib/s2s-engine/`). I built and maintain:

- **CSV Parser:** Handles X-Plane/FlyWithLua telemetry exports, strips duplicate headers, resamples to 1Hz via linear interpolation
- **Phase Detector:** Classifies flight segments (taxi, takeoff, climb, downwind, base, final, landing, touch-and-go, stall, steep turn) using state machines and altitude/speed thresholds
- **Per-Lesson Graders:** L1–L4 have full grading logic. Each grades specific maneuvers with tolerances defined by the rubric. L5–L17 return stubs until doctrine is provided.
- **Go-Around Detection (L2):** Detects GLGL events from telemetry, scores each on Power/Pitch/Flap/Climb categories, flags safety failures, generates coaching
- **Output Formatters:** JSON serializer (matching Python schema for cross-compatibility) and styled HTML debrief generator

C. Build the Full-Stack Web Application

LAYER

WHAT I BUILD

KEY TECH

Database	Schema design, migrations, seed scripts. Models for users, lessons, sessions, scores, quizzes, ground school content, progress tracking.	Prisma ORM, PostgreSQL
Authentication	Email/password login, role-based access (STUDENT vs ADMIN), session management, route protection.	NextAuth.js, JWT, bcrypt
API Layer	20+ REST endpoints: lesson CRUD, session processing, quiz submission, progress gating, admin operations, file upload presigning, ground school content delivery.	Next.js API Routes
File Storage	Telemetry CSV upload to cloud storage via presigned URLs. Client-side upload for performance.	S3, presigned URLs
Frontend	Dashboard (Path View), lesson detail pages (5-tab layout: Brief, Flight, Ground School, Quiz, Debrief), admin panel, login/signup flows.	React, Tailwind, Framer Motion
Data Viz	Altitude/speed charts, flight path plots, control input timelines, phase breakdown displays, GLGL event trackers.	Recharts

D. Implement Gate Logic & Progression

Students can't just click through lessons. I built and enforce:

- **Flight score gate:** Must score $\geq 3/5$ on telemetry grading
- **Quiz gate:** Must pass quiz (4/5 correct for Arc 1)
- **Debrief gate:** Must view the debrief to unlock the "Reveal" (theory explanation)
- **GLGL gate (L2):** Must complete 2 go-arounds + 2 landings with no safety failures
- **Check flight gates (L4/L7/L10/L14):** Must pass arc-boundary quiz before advancing to next arc
- **Admin bypass:** Admin role skips all gates for testing/support

E. Content Pipeline Management

I manage the pipeline from raw doctrine to live content:

- **Ingest:** Read uploaded JSON files (doctrine, rubrics) and Python reference code
- **Transform:** Convert to TypeScript modules, database records, and UI components
- **Seed:** Write and maintain seed scripts that safely upsert data without destroying production records
- **Validate:** Ensure readiness flags are consistent (groundSchoolReady, quizReady, scoringReady) so the UI reflects actual content availability
- **Version:** Track all changes via Story IDs (ST-####) with code markers for traceability

3. What a Typical Work Session Looks Like

Example: Wiring L2 Go-Around (ST-802D)

The owner uploads go-around doctrine JSON + Python scoring reference + rubric thresholds. Here's what I do:

1. **Read all inputs:** Parse the doctrine JSON (mental model, triggers, step-by-step), the Python scoring code (detect_go_around, score_go_around), and the rubric (thresholds for each category).
2. **Port Python → TypeScript:** Translate go_around.py into maneuvers/go-around.ts, preserving every threshold, formula, and edge case.
3. **Wire into grader:** Modify arc1-grader.ts so gradeL2() calls the new go-around detection and scoring, builds the GLGL event structure.
4. **Plumb through API:** Update sessions route to lift GLGL data into the response. Update progress logic to require GLGL pass.
5. **Build UI components:** Create GLGLTracker (4-event grid with safety banner), L2Doctrine (mental model cards), update Brief/Debrief tabs.
6. **Update lesson content:** Rewrite L2 objectives, brief, flight flow, common mistakes in both the Master Syllabus JSON and the seed script.
7. **Re-seed database:** Run seed scripts to push updated content to the live database.
8. **Test & checkpoint:** TypeScript compilation, build, runtime tests, browser verification, then save checkpoint.

4. Architecture I Maintain

Scoring Engine (lib/s2s-engine/)

constants.ts	V-speeds, KHMP data, tolerances
rubrics.ts	Result types, scoring utilities
telemetry-parser.ts	CSV parse + 1Hz resample

API Routes (app/api/)

/sessions	Upload + score flights
/lessons	Lesson list + gate status
/quiz-seeds/*	Quiz delivery + grading
/ground-school/*	Doctrine content

<code>phase-detector.ts</code>	Flight phase classification
<code>arc1-grader.ts</code>	L1-L4 grading + GLGL
<code>maneuvers/go-around.ts</code>	GA detect + score + coach
<code>json-formatter.ts</code>	Python-compatible serializer
<code>html-generator.ts</code>	Styled debrief HTML
<code>index.ts</code>	<code>evaluateFlight()</code> orchestrator

<code>/progress/*</code>	Debrief viewed, gates
<code>/admin/*</code>	Force complete, flags
<code>/upload/presigned</code>	S3 upload URLs
<code>/l2-doctrine</code>	Go-around doctrine
<code>/auth/*</code>	Login, signup, session

5. Rules I Follow

Code Discipline

- **No silent overwrites.** Every code change is wrapped in `// START ST-### Logic / // END ST-### Logic` markers so changes are traceable.
- **Risk assessment before touching core logic.** Low risk (CSS) = proceed. Mid risk (new functions) = request consent. High risk (schema/physics) = strict consent required.
- **Seed scripts use upsert, never delete.** Production data is sacred. No `--accept-data-loss` without explicit permission.
- **Test before checkpoint.** Every change goes through TypeScript check → build → runtime test → checkpoint. No exceptions.

Aviation Accuracy

- **C172 speeds only.** Vr=55, Vy=74, Vdownwind=90, Vapproach=65, Vglide=68. No inventing numbers.
- **1-5 score scale.** Labels: Rough / Developing / Solid / Sharp / Nailed it. No percentages.
- **Hub Rule.** Every lesson starts where the last ended. No teleporting between airports.
- **Physics over feelings.** Grading is based on resampled telemetry data, not subjective assessment.

Project Management

- **Story IDs.** Every task has a unique ST-#### or BL-#### identifier.
- **Backlog visibility.** Every major response includes a sprint table (ID, Status, Intent, Complexity).
- **Path View protocol.** Every new feature must describe how it affects the student's visual progression map. If it doesn't fit the path, it doesn't get built.

6. Current Status — What's Built

FEATURE	STATUS	DETAILS
Authentication	Live	Email/password login, STUDENT + ADMIN roles, JWT sessions
17-Lesson Curriculum	Live	All lessons seeded with descriptions, objectives, brief content
Path View (Dashboard)	Live	Arc-grouped lesson cards, status indicators, GLGL badge for L2
Telemetry Upload + Scoring	Live	CSV upload → S3 → server-side grading → debrief. L1-L4 graded.
Phase Detection	Live	Taxi, takeoff, climb, downwind, base, final, landing, T&G, stall, steep turn
GLGL Tracking (L2)	Live	Go-around detection, 4-event tracker, safety banner, pass conditions
Ground School (Arc 1)	Live	L1-L4 full doctrine: before you fly, mission, flight flow, mistakes, reveal
Quiz System (Arc 1)	Live	5 questions per lesson (L1-L4), 4/5 to pass, gates progression
Debrief Visualization	Live	Charts (altitude, speed, path, controls), phase breakdown, coaching
Gate Logic	Live	Score ≥ 3 + quiz pass + debrief viewed + GLGL (L2) = unlock next
Admin Panel	Live	Student monitoring, force-complete, readiness flag toggles
L5-L17 Scoring	Stub	Return LESSON_NOT_IMPLEMENTED until doctrine provided
Arc 2-5 Ground School	Pending	Waiting for curriculum content

7. What I Don't Do

- **Write aviation doctrine from scratch.** I implement what the owner provides. I don't invent lesson plans or grading rubrics.
- **Modify scoring math without consent.** If the Python says ± 5 knots tolerance, I use ± 5 . No "improvements."
- **Deploy without testing.** Every change goes through the full test pipeline before it hits production.
- **Break existing lessons.** When adding L2 GLGL, L1/L3/L4 must remain untouched. Regression = failure.
- **Make up data.** No placeholder images, fake scores, or invented telemetry. If content isn't provided, the UI shows "Placeholder" status.